Computer Vision:

Homework-3

Bilal Ahmed
ahmedb@purdue.edu

Metric Rectification

The goal of metric rectification is to remove projective and affine distortions in an image. We can use point-to-point correspondence or information of orthogonal lines in the image to compute the homography. We can then use this homography to correct the above mentioned distortions in the image.

Point-to-point correspondence

Let us pick a points $(x, y) \in \mathbb{R}^2$ on the domain image and its corresponding point on the range image $(x', y') \in \mathbb{R}^2$. We can use the homogeneous coordinate representation of these points to compute the homography matrix. HC representations of these points would be $\begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$ and $\begin{bmatrix} x'_1 \\ x'_2 \\ x'_3 \end{bmatrix}$ respectively. Mapping from point in the domain space to point in the range space would be given as;

$$\begin{bmatrix} x'_1 \\ x'_2 \\ x'_3 \end{bmatrix} = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & h_{33} \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ x_3 \end{bmatrix}$$

Writing the matrix-vector multiplication in the algebraic form;

$$x'_1 = h_{11}x_1 + h_{12}x_2 + h_{13}x_3$$
$$x'_2 = h_{21}x_1 + h_{22}x_2 + h_{23}x_3$$
$$x'_3 = h_{31}x_1 + h_{32}x_2 + h_{33}x_3$$

Let's get the expression for the physical coordinates (x', y'), using $x' = \frac{x'_1}{x'_3}$ and $y' = \frac{x'_2}{x'_3}$. So we have,

$$x' = \frac{x'_1}{x'_3} = \frac{h_{11}x_1 + h_{12}x_2 + h_{13}x_3}{h_{31}x_1 + h_{32}x_2 + h_{33}x_3}$$

and

$$y' = \frac{x'_2}{x'_3} = \frac{h_{21}x_1 + h_{22}x_2 + h_{23}x_3}{h_{31}x_1 + h_{32}x_2 + h_{33}x_3}$$

Dividing numerator and denominator of both the equations by x_3 gives us physical coordinates (x,y) on the right hand side as well.

$$x' = \frac{h_{11}x + h_{12}y + h_{13}}{h_{31}x + h_{32}y + h_{33}}$$
$$y' = \frac{h_{21}x + h_{22}y + h_{23}}{h_{31}x + h_{32}y + h_{33}}$$

Rearranging,

$$h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx' - h_{33}x' = 0$$
$$h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy' - h_{33}y' = 0$$

Since H is also homogeneous, meaning what matters is the rations of its components, we can set $h_{33} = 1$. In that case the above expressions become;

$$h_{11}x + h_{12}y + h_{13} - h_{31}xx' - h_{32}yx' = x'$$
$$h_{21}x + h_{22}y + h_{23} - h_{31}xy' - h_{32}yy' = y'$$

A pair of corresponding points in both planes gives us 2 equations, since we need to learn 8 parameters, we'd need 4 such pairs. Using 4 pairs of points, we'd have 8 equations;

$$h_{11}x_1 + h_{12}y_1 + h_{13} - h_{31}x_1x_1' - h_{32}y_1x_1' = x_1'$$
$$h_{21}x_1 + h_{22}y_1 + h_{23} - h_{31}x_1y_1' - h_{32}y_1y_1' = y_1'$$
$$h_{11}x_2 + h_{12}y_2 + h_{13} - h_{31}x_2x_2' - h_{32}y_2x_2' = x_2'$$
$$h_{21}x_2 + h_{22}y_2 + h_{23} - h_{31}x_2y_2' - h_{32}y_2y_2' = y_2'$$
$$h_{11}x_3 + h_{12}y_3 + h_{13} - h_{31}x_3x_3' - h_{32}y_3x_3' = x_3'$$
$$h_{21}x_3 + h_{22}y_3 + h_{23} - h_{31}x_3y_3' - h_{32}y_3y_3' = y_3'$$
$$h_{11}x_4 + h_{12}y_4 + h_{13} - h_{31}x_4x_4' - h_{32}y_4x_4' = x_4'$$
$$h_{21}x_4 + h_{22}y_4 + h_{23} - h_{31}x_4y_4' - h_{32}y_4y_4' = y_4'$$

We can write these 8 set of equations in the matrix form;

$$\begin{bmatrix} x_1 & y_1 & 1 & 0 & 0 & 0 & -x_1x_1' & -y_1x_1' \\ 0 & 0 & 0 & x_1 & y_1 & 1 & -x_1y_1' & -y_1y_1' \\ x_2 & y_2 & 1 & 0 & 0 & 0 & -x_2x_2' & -y_2x_2' \\ 0 & 0 & 0 & x_2 & y_2 & 1 & -x_2y_2' & -y_2y_2' \\ x_3 & y_3 & 1 & 0 & 0 & 0 & -x_3x_3' & -y_3x_3' \\ 0 & 0 & 0 & x_3 & y_3 & 1 & -x_3y_3' & -y_3y_3' \\ x_4 & y_4 & 1 & 0 & 0 & 0 & -x_4x_4' & -y_4x_4' \\ 0 & 0 & 0 & x_4 & y_4 & 1 & -x_4y_4' & -y_4y_4' \end{bmatrix} \begin{bmatrix} h_{11} \\ h_{12} \\ h_{13} \\ h_{21} \\ h_{22} \\ h_{23} \\ h_{31} \\ h_{32} \end{bmatrix} = \begin{bmatrix} x_1' \\ y_1' \\ x_2' \\ y_2' \\ x_3' \\ y_3' \\ x_4' \\ y_4' \end{bmatrix}$$

or we can simply write this as;

$$A\boldsymbol{x} = \boldsymbol{y}$$

So, given 4 corresponding pairs of points, we can solve this set of linear equations using **np.linalg.lstsq**, to get $\boldsymbol{x}$. Once we get $\boldsymbol{x}$, homographic matrix H would be;

$$H = \begin{bmatrix} h_{11} & h_{12} & h_{13} \\ h_{21} & h_{22} & h_{23} \\ h_{31} & h_{32} & 1 \end{bmatrix}$$

Applying homography H to the distorted image, we can get the image without projective and affine distortion.

Two-step method

In the two-step method, we first use the vanishing line (VL) to remove projective transform, and then we use the information about orthogonal lines (in the un-distorted image) to get rid of affine distortion. Any two straight lines will intersect each other at one point. In case of parallel lines, they intersect each other at a point at infinity. In case of projective distortion, parallel lines intersect at vanishing points, in the space of image. A vanishing line (VL) is a line that connects two vanishing points of an image. We can remove projective distortion in an image, using a transform that sends vanishing line back to infinity, $l_\infty = \begin{bmatrix}0\\0\\1\end{bmatrix}$. So we use two pairs of parallel lines to compute homogeneous coordinates of vanishing lines $\begin{bmatrix}l_1\\l_2\\l_3\end{bmatrix}$. Once we get the vanishing line, we can remove projective distortion from an image, by applying the homography given as follows;

$$H = \begin{bmatrix}1 & 0 & 0\\0 & 1 & 0\\l_1 & l_2 & l_3\end{bmatrix}$$

.

After we get rid of projective distortion we can use the expression of $cos(\theta)$, where θ is the angle between orthogonal lines in the distorted image. Setting this angle $\theta = 90$, we can compute homography that gets rid of affine distortion.

$$cos(\theta) = \mathbf{l'}^T\mathbf{H}\mathbf{C}^*_\infty\mathbf{H}^T\mathbf{m'} = 0$$

where; $H = \begin{bmatrix}\mathbf{A} & \mathbf{t}\\\mathbf{0}^T & 1\end{bmatrix}$ and $\mathbf{C}^*_\infty = \begin{bmatrix}1 & 0 & 0\\0 & 1 & 0\\0 & 0 & 0\end{bmatrix}$.

$$\begin{bmatrix}l'_1 & l'_2 & l'_3\end{bmatrix}\begin{bmatrix}A & t\\0^\top & 1\end{bmatrix}\begin{bmatrix}I & 0\\0^\top & 0\end{bmatrix}\begin{bmatrix}A^\top & 0\\t^\top & 1\end{bmatrix}\begin{bmatrix}m'_1\\m'_2\\m'_3\end{bmatrix} = 0$$

$$\begin{bmatrix}l'_1 & l'_2 & l'_3\end{bmatrix}\begin{bmatrix}AA^\top & 0\\0^\top & 0\end{bmatrix}\begin{bmatrix}m'_1\\m'_2\\m'_3\end{bmatrix} = 0$$

Let $S = AA^\top$, then

$$\begin{bmatrix}l'_1 & l'_2\end{bmatrix}^\top\begin{bmatrix}S_{11} & S_{12}\\S_{21} & S_{22}\end{bmatrix}\begin{bmatrix}m'_1\\m'_2\end{bmatrix} = 0$$

Since $AA^\top$ is symmetric, $S_{12} = S_{21}$. Since we are only interested in ratios, we can set $S_{21} = 1$. Therefore,

$$S_{11}l'_1m'_1 + S_{12}l'_1m'_2 + S_{12}l'_2m'_1 + l'_2m'_2 = 0$$

or

$$S_{11}l'_1m'_1 + S_{12}(l'_1m'_2 + l'_2m'_1) = -l'_2m'_2$$

. One angle correspondence gives us one equation, we can get another equation using 2nd angle-to-angle correspondence. We can write two equation in the marix form as;

$$\begin{bmatrix}l'_1m'_1 & (l'_1m'_2 + l'_2m'_1)\\p'_1q'_1 & (p'_1q'_2 + p'_2q'_1)\end{bmatrix}\begin{bmatrix}S_{11}\\S_{12}\end{bmatrix} = \begin{bmatrix}-l'_2m'_2\\-p'_2q'_2\end{bmatrix}$$

Where p and q are second pair of lines, giving us second correspondence. Solving this would give us matrix S, then we can use eigen decomposition to get A matrix, as follow;

$$S = AA^\top$$

$$= VDV^\top VDV^\top$$
$$= VD^2V^\top$$
$$= V\begin{bmatrix}\lambda_1^2 & 0\\ 0 & \lambda_2^2\end{bmatrix}V^\top$$

At this point, we have the affine homography matrix H, we can use this to get rid of affine distortion.

One-step method

In one step method, instead of using vanishing line, we use the transformation of degenerate conic to get the general homography that corrects for both projective and affine distortions at the same time. Since,

$$C'^*_\infty = HC^*_\infty H^\top$$

we have,

$$\cos(\theta) = \mathbf{l}'^\top C'^*_\infty \mathbf{m}' = 0$$

where;

$$C'^* = \begin{bmatrix}a & \frac{b}{2} & \frac{d}{2}\\ \frac{b}{2} & c & \frac{e}{2}\\ \frac{d}{2} & \frac{e}{2} & f\end{bmatrix}$$

Therefor, the expression becomes;

$$\begin{bmatrix}l'_1 & l'_2 & l'_3\end{bmatrix}\begin{bmatrix}a & \frac{b}{2} & \frac{d}{2}\\ \frac{b}{2} & c & \frac{e}{2}\\ \frac{d}{2} & \frac{e}{2} & f\end{bmatrix}\begin{bmatrix}m'_1\\ m'_2\\ m'_3\end{bmatrix} = 0$$

$$l'_1m'_1a + \frac{l'_2m'_1 + l'_1m'_2}{2}b + l'_2m'_2c + \frac{l'_1m'_3 + l'_3m'_1}{2}d + \frac{l'_3m'_2 + l'_2m'_3}{2}e + l'_3m'_3f = 0$$

Setting f=1, since we are only interested in ratios, we have one equation with 5 unknowns. So we need total 5 pair of corresponding lines (orthogonal in the undistorted image) to solve for 5 unknowns. After this, we can solve homography H as follows;

$$C'^* = HC^*_\infty H^\top$$
$$= \begin{bmatrix}A & 0\\ v^\top & 1\end{bmatrix}\begin{bmatrix}1 & 0 & 0\\ 0 & 1 & 0\\ 0 & 0 & 0\end{bmatrix}\begin{bmatrix}A^\top & v\\ 0^\top & 1\end{bmatrix}$$
$$= \begin{bmatrix}AA^\top & Av\\ v^\top A & v^\top v\end{bmatrix}$$
$$AA^\top = \begin{bmatrix}a & \frac{b}{2}\\ \frac{b}{2} & c\end{bmatrix}$$

Again, as we did in the previous section, we can use eigen decomposition to get the matrix A, and using A^{-1}, we can get v as;

$$v = A^{-1}\begin{bmatrix}\frac{d}{2}\\ \frac{e}{2}\end{bmatrix}$$

Then using $H = \begin{bmatrix}\mathbf{A} & \mathbf{0}\\ \mathbf{v}^T & 1\end{bmatrix}$ we have the homography matrix, and inverse of this will be used to correct the distorted image.

Results

Task-1: Tension board

The distorted image of tension board is shown in Fig.1a along with set of points PQRS used for transformation highlighted in the image. Coordinates of points PQRS are also given in table 2. All correction approaches explained above were used and results are displayed in Here are the homography matrices I got;

Image Type	P	Q	R	S
Board	(420, 65)	(140, 1217)	(1958, 1356)	(1794, 421)

Table 1: Coordinates of points P, Q, R, and S for chosen sets of image points.

$$H_{p2p} = \begin{bmatrix} 6.46 \times 10^{-3} & 1.57 \times 10^{-3} & -2.81 \\ -2.31 \times 10^{-3} & 8.93 \times 10^{-3} & 3.92 \\ -2.24 \times 10^{-4} & 2.65 \times 10^{-4} & 1.08 \end{bmatrix}$$

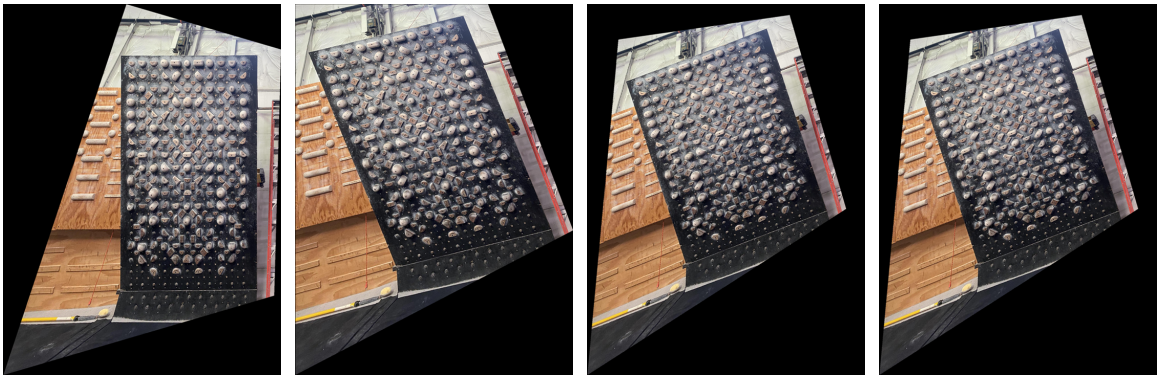
H_{proj} is the homography matrix that uses only projective distortions using vanishing line, H_{aff} remove affine distortions and H_{comb} combines both corrections and is obtained by matrix multiplication $H_{comb} = H_{proj} \times H_{aff}$

$$H_{proj} = \begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ -2.08 \times 10^{-4} & 2.46 \times 10^{-4} & 1.00 \end{bmatrix}$$

$$H_{aff} = \begin{bmatrix} 0.69 & -0.10 & 0.00 \\ -0.10 & 1.02 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

(a) Tension Board image (b) Corridor image

Figure 1: Given (distorted) images of board and corridor with choosen points PQRS marked in each image



(a) point-to-point (b) two-point projective (c) two-point proj & affine (d) one-point

Figure 2: Transformed images of $board_1$ (with type of transformation mentioned in the sub-caption)

(a) Ipad (b) Photo frame

Figure 3: Distorted images of IPad screen and photo frame with choosen points PQRS marked in each image

$$H_{comb} = \begin{bmatrix} 6.91 \times 10^{-1} & -9.69 \times 10^{-2} & 0.00 \\ -9.69 \times 10^{-2} & 1.02 & 0.00 \\ -1.68 \times 10^{-4} & 2.72 \times 10^{-4} & 1.00 \end{bmatrix}$$

H_{1step} is the homography matrix obtained using one-step method that corrects for both projective and affine distortions.

$$H_{1step} = \begin{bmatrix} 2.10 \times 10^{-4} & -4.98 \times 10^{-5} & 2.54 \times 10^{-21} \\ -4.98 \times 10^{-5} & 4.26 \times 10^{-4} & -1.17 \times 10^{-20} \\ -2.08 \times 10^{-4} & 2.46 \times 10^{-4} & 1.00 \end{bmatrix}$$

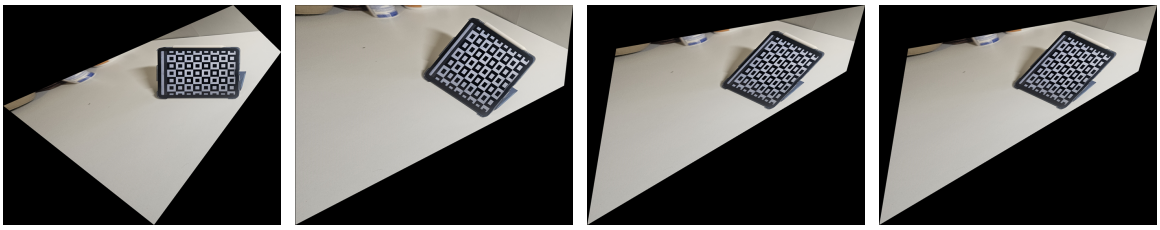
Task-2: Ipad

For task 2, I have used two images of my own, shown in Fig. 3. For these two images as well, I apply the methods explained above. For Ipad images, the corrected images are displayed in Fig. 4. I have also given the transformation matrices here.

$$H_{p2p} = \begin{bmatrix} 1.83 \times 10^{-2} & -2.06 \times 10^{-2} & 4.68 \\ 1.63 \times 10^{-2} & 3.20 \times 10^{-2} & -10.28 \\ -6.70 \times 10^{-6} & 1.36 \times 10^{-3} & 0.61 \end{bmatrix}$$

Image Type	P	Q	R	S
IPad	(67, 287)	(325, 516)	(524, 354)	(260, 189)
Frame	(143, 391)	(220, 628)	(460, 352)	(276, 106)

Table 2: Coordinates of points P, Q, R, and S for chosen sets of image points.



(a) point-to-point (b) two-point projective (c) two-point proj & affine (d) one-point

Figure 4: Transformed images of *ipad* (with type of transformation mentioned in the sub-caption)

$$H_{proj} = \begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ -1.10 \times 10^{-5} & 2.23 \times 10^{-3} & 1.00 \end{bmatrix}$$

$$H_{aff} = \begin{bmatrix} 0.77 & -0.15 & 0.00 \\ -0.15 & 1.05 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

$$H_{comb} = \begin{bmatrix} 7.68 \times 10^{-1} & -1.53 \times 10^{-1} & 0.00 \\ -1.53 \times 10^{-1} & 1.05 & 0.00 \\ -3.51 \times 10^{-4} & 2.35 \times 10^{-3} & 1.00 \end{bmatrix}$$

$$H_{1step} = \begin{bmatrix} 9.60 \times 10^{-4} & -6.02 \times 10^{-5} & 4.43 \times 10^{-18} \\ -6.02 \times 10^{-5} & 2.24 \times 10^{-3} & -9.49 \times 10^{-17} \\ -1.10 \times 10^{-5} & 2.23 \times 10^{-3} & 1.00 \end{bmatrix}$$

Task-2: Frame

This is continuation of task 2, and its results are given in Fig. 5, here are the matrices computed for this section.

$$H_{p2p} = \begin{bmatrix} 8.10 \times 10^{-2} & -2.63 \times 10^{-2} & -1.29 \\ 7.42 \times 10^{-2} & 3.46 \times 10^{-2} & -24.16 \\ 3.34 \times 10^{-3} & 1.36 \times 10^{-4} & 4.69 \times 10^{-1} \end{bmatrix}$$

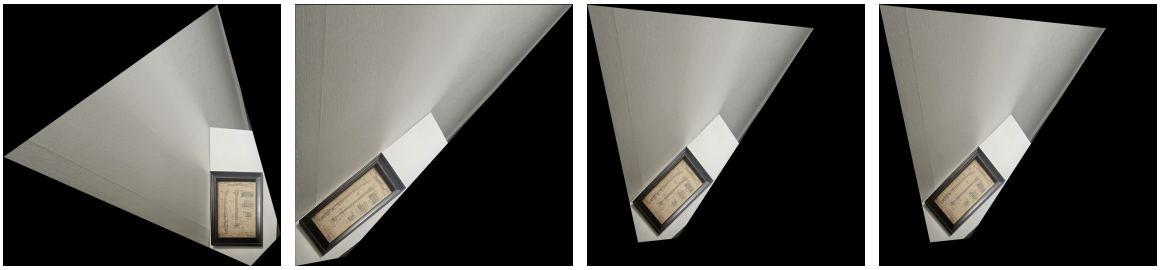
$$H_{proj} = \begin{bmatrix} 1.00 & 0.00 & 0.00 \\ 0.00 & 1.00 & 0.00 \\ 7.13 \times 10^{-3} & 2.91 \times 10^{-4} & 1.00 \end{bmatrix}$$

$$H_{aff} = \begin{bmatrix} 2.54 & 0.23 & 0.00 \\ 0.23 & 1.03 & 0.00 \\ 0.00 & 0.00 & 1.00 \end{bmatrix}$$

here

$$H_{comb} = \begin{bmatrix} 2.54 \times 10^{0} & 2.31 \times 10^{-1} & 0.00 \times 10^{0} \\ 2.31 \times 10^{-1} & 1.03 \times 10^{0} & 0.00 \times 10^{0} \\ 1.82 \times 10^{-2} & 1.94 \times 10^{-3} & 1.00 \times 10^{0} \end{bmatrix}$$

$$H_{1step} = \begin{bmatrix} 8.67 \times 10^{-3} & 1.00 \times 10^{-3} & -2.93 \times 10^{-17} \\ 1.00 \times 10^{-3} & 1.23 \times 10^{-3} & -3.88 \times 10^{-18} \\ 7.13 \times 10^{-3} & 2.91 \times 10^{-4} & 1.00 \times 10^{0} \end{bmatrix}$$



(a) point-to-point (b) two-point projective (c) two-point proj & affine (d) one-point

Figure 5: Transformed images of $frame$ (with type of transformation mentioned in the sub-caption)

Discussion

Different approaches for getting rid of projective and affine distortions were compared. They all have their own pros and cons, like point-to-point needs slightly more work in terms of getting point correspondences. But I think results suggest point-to-point gives good results once the correspondences are established. I tried a lot of different combination of points for corridor image, but that image is very difficult to correct. As I computed the homography matrix, and checked its sub-matrix A, it turns out that sub- matrix is not positive definite. So as a result as I apply that to the image, it has some compicated reflections of pixel coordinates. I did try to view only the region within the selected four points, that did work alright (I have not included that figure in the report). In summary, for slight distortions the aproaches worked fine and one-step method is the most convenient to use, but we need 5 pairs of orthogonal lines for that. Two-point method needed only four points but it's drawback is that is needs an extra step. Point-to-point needs point correspondences but it works very good.

Python Code

```python
import cv2
import pathlib
import skimage
import computer_vision

import matplotlib.pylab as plt
import matplotlib as mpl
import numpy as np

def compute_general_homography(point_pairs):
    """Takes 4 pairs of points in two images, where
    each pair of points is a pixel location in one image and corresponding
    pixel location in the 2nd image.
    Computes general planar projective transform that maps pixel locations
    in the image 1 to pixel locations in the image 2.

    Args:
        point_pairs: list(list(tuples)): Each pair of point is represented
            as [(x,y), (x', y')]. Function expects 4 such pairs of points.
    """
    num_points = len(point_pairs)
    assert (num_points) >=4, f"expects at least 4 pairs of points but got {num_points}"

    A = []
    y = []
    # np.array([x_p1, y_p1, x_p2, y_p2, x_p3, y_p3, x_p4, y_p4])[:, None]
    for i, ((x1, y1), (x_p1, y_p1)) in enumerate(point_pairs):
        A.append([x1, y1, 1, 0, 0, 0, -x1*x_p1, -y1*x_p1])
        A.append([0, 0, 0, x1, y1, 1, -x1*y_p1, -y1*y_p1])
        y.extend([x_p1, y_p1])

    A = np.array(A)
    y = np.array(y)[:,None]

    h = np.linalg.lstsq(A, y)[0]
    h = np.append(h, 1)
    # homography matrix would be...
    H = h.reshape(3,3)
    return H

def transform_pixel_coordinate(H, pixel_coordinates):
    """Applies homography H to the 2D pixel coordinates
    and returns transformed 2D pixel coordinates.

    Args:
        H: shape (3, 3): Homography matrix
        pixel_coordinates: shape (2, n): n being the number of coordinates.
    """
    # pixel_hc = np.concatenate([pixel_coordinates, np.ones(pixel_coordinates.shape[1])[None, :]], axis=0)
    pixel_hc = np.array([*pixel_coordinates, 1])
    transformed_hc = np.matmul(H, pixel_hc) # gives me (3,n)
    transformed_hc /= transformed_hc[-1]#[None, :]
    return transformed_hc[:-1]

```

```python
def read_image(subdir, image_name):
    """Reads images from the subdir"""
    path = pathlib.Path(computer_vision.__path__[0])
    image_path = path.parents[0] / 'images' /subdir/image_name
    img_bgr = cv2.imread(image_path)
    # Convert the image from BGR to RGB format
    img_rgb = cv2.cvtColor(img_bgr, cv2.COLOR_BGR2RGB)
    return img_rgb

def save_image(image, subdir, image_name):
    """Saves image to the subdir"""
    path = pathlib.Path(computer_vision.__path__[0])
    image_path = path.parents[0] / 'images' /subdir/image_name
    # plt.imshow(image)
    # plt.savefig(image_path)
    # Save the image with specific quality
    img_bgr_for_saving = cv2.cvtColor(image, cv2.COLOR_RGB2BGR)
    cv2.imwrite(image_path, img_bgr_for_saving)

def get_physical_coordinates(x_hc):
    x = x_hc[0]/x_hc[2]
    y = x_hc[1]/x_hc[2]
    return (x,y)

def get_homogeneous_coordinates(X):
    return [*X, 1]

def get_output_space_dimensions(H, img):
    """Map the corners of the image to the physical coordinates,
    to get the extreme coordinates in the physical coordiantes,
    Use these extreme points to adjust physical coordinates,
    otherwise everything will be mapped to within the grid, that was used
        for
    dimensions in the physical coordinates initially.

    Args:
        H: 3x3 matrix = Homography matrix
        img: ndarray

    Returns:
        physical_height
        physical_width
        offset_height
        offset_width
    """
    img_height, img_width, _ = img.shape
    print(f"Input image dim: {img_height}x{img_width}")
    image_corners = [
        [0,0], [0, img_width], [img_height, img_width], [img_height, 0]
    ]
    output_corners = []
    for point in image_corners:
        output_corners.append(transform_pixel_coordinate(H, point))
    output_corners = np.stack(output_corners, axis=0)

    bottom, right = np.max(output_corners, axis=0)
    top, left = np.min(output_corners, axis=0)
    output_height = bottom - top
    output_width = right - left
```

```
113
114     print(f"Output image dim: {output_height}x{output_width}")
115     print(f"Output coordinates offsets: {top}, {left}")
116     return output_height, output_width, top, left
117
118 def is_A_positive_definite(H):
119     """Checks if sub-matrix A (within H) is positive definite of not"""
120     return bool(H[0,0]>0 and H[0,0]*H[1,1] > H[0,1]*H[1,0])
121
122 def apply_homography(H, img, resize=True, output_dim=(1200, 800)):
123     """Takes in homography matrix 'H' and input image 'img'
124     and returns output image obtained after application of homography.
125
126     Args:
127         vectorize: bool= Default=True, vectorize homography application.
128     """
129     if resize:
130         H, *_ = adjust_homography_matrix(H, img, output_dim)
131     output_height, output_width, height_offset, width_offset =
            get_output_space_dimensions(H, img)
132     img_height, img_width, _ = img.shape
133     H_inv = np.linalg.pinv(H)
134     output_height = int(output_height+0.5)
135     output_width = int(output_width+0.5)
136     print(f"creating image size: {output_height}x{output_width}")
137     output_img = np.zeros((output_height, output_width, 3), dtype=img.dtype)
138     pixel_y, pixel_x = np.meshgrid(range(output_height), range(output_width)
            )
139     hc_coordinates = np.stack([pixel_y.flatten(), pixel_x.flatten(), np.
            ones_like(pixel_x).flatten()])
140     transformed_coordinates = np.matmul(H_inv, hc_coordinates)
141     transformed_coordinates /= transformed_coordinates[-1]
142     transformed_coordinates = transformed_coordinates.astype(int)
143     # create masks for valid (within img range) coordinates...
144     mask_coordinates = (transformed_coordinates[0] > 0) & (
            transformed_coordinates[0] < img_height) & \
145             (transformed_coordinates[1] > 0) & (transformed_coordinates[1] <
                    img_width)
146
147     output_img[hc_coordinates[:,mask_coordinates][0], hc_coordinates[:,
            mask_coordinates][1]] = img[
148         transformed_coordinates[:,mask_coordinates][0],
149         transformed_coordinates[:,mask_coordinates][1]
150         ]
151     return output_img
152
153
154
155 def adjust_homography_matrix(H, img, output_dim):
156     output_height, output_width, height_offset, width_offset =
            get_output_space_dimensions(H, img)
157     # img_height, img_width, _ = img.shape
158     # H_inv takes us back from output space to input space...
159         # translate coordinates...
160     translation_vector = [-1*height_offset, -1*width_offset]
161     H_translate = np.eye(3)
162     H_translate[:2, 2] = translation_vector
163     H = np.matmul(H_translate, H)
164     H_resize = np.eye(3)
```

```
    H_resize[0,0] = output_dim[0]/output_height
    H_resize[1,1] = output_dim[1]/output_width
    output_height = output_dim[0]
    output_width = output_dim[1]
    H = np.matmul(H_resize, H)
    return H, output_height, output_width


def correct_distortion_using_point_to_point_correspondence(
        img,
        physical_points,
        image_points,
        vectorize=True,
        resize=True,
        output_dim=(1200,1000),

    ):
    """Uses point to point correspondence to compute homography,
    and correct purely projective and affine distortions in the image.

    Args:
        img: input (distorted) image
        physical_points: points on the physical plane
        image_points: points on the image plane
        scale_factor: float = factor that maps physical measurements to
            pixles
    """
    # physical_points = [(p1*scale_factor, p2*scale_factor) for p1,p2 in
        physical_points]
    point_pairs = [[p1, p2] for p1, p2 in zip(physical_points, image_points)
        ]

    H = compute_general_homography(point_pairs)
    if not is_A_positive_definite(H):
        H = condition_matrix(H)
    #     raise RuntimeError("sub-matrix A is not positive definit. Try
        choosing different points")

    # H = condition_matrix(H)
    H_inv = np.linalg.pinv(H)
    print(f"H_p2p: \nH_inv")
    output_img = apply_homography(
        H_inv, img, vectorize=vectorize, resize=resize, output_dim=
            output_dim)

    return output_img, H_inv



def condition_matrix(H):
    """Homography matrices can have high condition number,
    that can make the transform very sensitive,
    This function improves condition number of H by adding
    small perturbations to the singular values.
    """
    U, S, Vt = np.linalg.svd(H)
    epsilon = 1e-1
    # S[np.where(S<epsilon)] = epsilon
    # S_perturbed = S
```

```
    S_perturbed = S + epsilon*np.random.rand(S.size)
    # S_perturbed = S + np.random.rand(S.size)
    D = S_perturbed*np.eye(3)
    conditioned_H = U@D@Vt
    # epsilon = 1e-3  # Small scalar for noise magnitude
    # noise = epsilon * np.random.randn(3, 3)
    # conditioned_H = H + noise
    return conditioned_H


def compute_H_proj(points):
    """Given points on the image, computes homography
    that removed purely projective distortions, using the
    vanishing line (VL).
    """
    P = get_homogeneous_coordinates(points[0])
    Q = get_homogeneous_coordinates(points[1])
    R = get_homogeneous_coordinates(points[2])
    S = get_homogeneous_coordinates(points[3])

    l1 = np.cross(P,S)
    l2 = np.cross(Q,R)
    vp1 = np.cross(l1, l2)

    l3 = np.cross(P,Q)
    l4 = np.cross(S,R)
    vp2 = np.cross(l3, l4)

    vl = np.cross(vp1, vp2)
    vl = vl.astype(np.float64) / vl[-1]

    H_proj = np.eye(3)
    H_proj[2,:] = vl
    return H_proj

def compute_H_aff(points):
    """Given points on a rectangle, computes the homography
    that removes affine distortion. Uses the expression for
    cos(angle) between lines that are expected to be orthogonal
    in the undistorted image.
    """
    P = get_homogeneous_coordinates(points[0])
    Q = get_homogeneous_coordinates(points[1])
    R = get_homogeneous_coordinates(points[2])
    S = get_homogeneous_coordinates(points[3])

    l1 = np.cross(P,S).astype(np.float64)
    m1 = np.cross(P,Q).astype(np.float64)
    l2 = np.cross(P,R).astype(np.float64)
    m2 = np.cross(Q,S).astype(np.float64)

    l1 /= l1[-1]
    m1 /= m1[-1]
    l2 /= l2[-1]
    m2 /= m2[-1]

    X = np.zeros((2,2))
    X[0,0] = l1[0]*m1[0]
    X[0,1] = l1[0]*m1[1] + l1[1]*m1[0]
```

```
    X[1,0] = l2[0]*m2[0]
    X[1,1] = l2[0]*m2[1] + l2[1]*m2[0]

    y = np.zeros((2,1))
    y[0] = -l1[1]*m1[1]
    y[1] = -l2[1]*m2[1]

    sol = np.linalg.lstsq(X, y)[0].squeeze()
    S = np.zeros((2,2))
    S[0,0] = sol[0]
    S[0,1] = sol[1]
    S[1,0] = sol[1]
    S[1,1] = 1

    e, V = np.linalg.eig(S)

    D = np.eye(2)*np.sqrt(e)
    A = V@D@V.T
    H_aff = np.eye(3)
    H_aff[:2, :2] = A
    return np.linalg.pinv(H_aff)


def correct_distortion_using_two_step_method(
        img,
        image_points,
        vectorize=True,
        resize=False,
        output_dim=(1200,1000),
    ):
    """Uses Vanishing line (VL) to remove purely projective
    distortion and then uses orthogonal lines to get the affine distortion.
    """
    H_proj = compute_H_proj(image_points)
    print(f"H_proj: \n{H_proj}")
    H_aff = compute_H_aff(image_points)
    print(f"H_aff: \n{H_aff}")
    H_comb = H_proj@H_aff
    print(f"H_comb: \n{H_comb}")

    img_proj = apply_homography(H_proj, img, vectorize=vectorize, resize=
        resize, output_dim=output_dim)
    # H_aff_inv = np.linalg.pinv(H_aff)
    img_aff = apply_homography(H_aff, img_proj, vectorize=vectorize, resize=
        resize, output_dim=output_dim)
    return img_proj, img_aff, H_proj, H_aff, H_comb

def get_line_pairs(corner_points):
    """Given the corner points of the rectangle,
    returns five line pairs
    """
    epsilon = 1e-6
    hc_points = [get_homogeneous_coordinates(pt) for pt in corner_points]
    line_pairs = []
    num_points = len(hc_points)
    for i in range(num_points):
        p1 = (i-1)%num_points
        p2 = i
        p3 = (i+1)%num_points
```

```
        l1 = np.cross(hc_points[p1], hc_points[p2]).astype(np.float64)
        l1 /= l1[-1] + epsilon
        l2 = np.cross(hc_points[p2], hc_points[p3]).astype(np.float64)
        l2 /= l2[-1] + epsilon
        line_pairs.append([l1, l2])

    p1, p2 = 0, 1
    l1 = np.cross(hc_points[p1], hc_points[(p1+2)%num_points]).astype(np.
        float64)
    l1 /= l1[-1] + epsilon
    l2 = np.cross(hc_points[p2], hc_points[(p2+2)%num_points]).astype(np.
        float64)
    l2 /= l2[-1] + epsilon
    line_pairs.append([l1, l2])
    return line_pairs

def compute_H_one_step(corner_points):
    """Given the five line pairs computes homography matrix,
    using one step method of using orthogonal lines."""

    # from the corner points, get the five pair of lines..
    line_pairs = get_line_pairs(corner_points)

    X = []
    y = []
    for l, m in line_pairs:
        row = [l[0]*m[0], (l[0]*m[1]+l[1]*m[0]), l[1]*m[1], (l[0]*m[2]+l[2]*
            m[0]), (l[1]*m[2]+l[2]*m[1])]
        X.append(row)
        y.append(-l[2]*m[2])
    X = np.array(X)
    y = np.array(y)[:, None]

    # least-squares solution to the system of equations..
    sol = np.linalg.lstsq(X, y)[0].squeeze()

    S = np.array([[sol[0], sol[1]],
                  [sol[1], sol[2]]])

    e, V = np.linalg.eig(S)
    D = np.eye(2)*np.sqrt(e)
    A = V@D@V.T
    v = np.linalg.pinv(A)@np.array([sol[3], sol[4]])[:, None]
    H = np.eye(3)
    H[:2, :2] = A
    H[2, :2] = v.T
    return np.linalg.pinv(H)




if __name__ == '__main__':


    # # Task-1: board image
    physical_points = [
        (0, 0),
        (0, 8),
        (12, 8),
```

```
        (12, 0),
    ]
    image_points = [
        (420, 65),
        (140, 1217),
        (1958, 1356),
        (1794, 421),
    ]
    # board image: Point-to-point
    filename = 'board_1.jpeg'
    img = read_image('hw3', filename)
    img_p2p, H = correct_distortion_using_point_to_point_correspondence(
        img,
        physical_points=physical_points,
        image_points=image_points,
        vectorize=True,
        resize=True,
        output_dim=img.shape[:-1],
        )

    method = 'point-to-point'
    save_image(img_p2p, 'hw3', f'{filename[:-5]}-{method}.png')

    # board image: two-step
    img_proj, img_aff, H_proj, H_aff, H_comb =
        correct_distortion_using_two_step_method(
        img, image_points, resize=True, output_dim=img.shape[:-1]
    )
    method = 'two-step-proj'
    save_image(img_proj, 'hw3', f'{filename[:-5]}-{method}.png')
    method = 'two-step-aff'
    save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')

    # board image: one-step
    H = compute_H_one_step(image_points)
    output_img = apply_homography(H, img, resize=True, output_dim=img.shape
        [:-1])
    method = 'one-step'
    save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')

    # Task2: corridor
    physical_points = [
        (0,0),
        (0,24),
        (8, 24),
        (8, 0),
    ]
    image_points = [
        (541, 533),
        (249, 1313),
        (1340, 1294),
        (909, 525)
    ]

    filename = 'corridor.jpeg'
    img = read_image('hw3', filename)

    # corridor: point-to-point
    img_p2p, H = correct_distortion_using_point_to_point_correspondence(
```

```
        img,
        physical_points=physical_points,
        image_points=image_points,
        vectorize=True,
        resize=True,
        output_dim=img.shape[:-1],
        )

    method = 'point-to-point'
    save_image(img_p2p, 'hw3', f'{filename[:-5]}-{method}.png')

    # corridor: two-step
    img_proj, img_aff, H_proj, H_aff, H_comb =
        correct_distortion_using_two_step_method(
        img, image_points, resize=True, output_dim=img.shape[:-1]
    )
    method = 'two-step-proj'
    save_image(img_proj, 'hw3', f'{filename[:-5]}-{method}.png')
    method = 'two-step-aff'
    save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')

    # corridor: one-step
    H = compute_H_one_step(image_points)

    output_img = apply_homography(H, img, resize=True, output_dim=img.shape
        [:-1])
    method = 'one-step'
    save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')

    # Task2: IPad
    # Ipad: point-to-point
    physical_points = [
        (0, 0),
        (0, 8.8),
        (6.4, 8.8),
        (6.4, 0),
    ]
    image_points = [
        (67, 287),
        (325, 516),
        (524, 354),
        (260, 189),
    ]

    filename = 'ipad.png'
    img = read_image('hw3', filename)

    img_p2p, H = correct_distortion_using_point_to_point_correspondence(
        img,
        physical_points=physical_points,
        image_points=image_points,
        vectorize=True,
        resize=True,
        output_dim=img.shape[:-1],
        )

    method = 'point-to-point'
    save_image(img_p2p, 'hw3', f'{filename[:-5]}-{method}.png')

```

```
    # Ipad: Two-point method...
    img_proj, img_aff, H_proj, H_aff, H_comb =
        correct_distortion_using_two_step_method(
        img, image_points, resize=True, output_dim=img.shape[:-1]
    )
    method = 'two-step-proj'
    save_image(img_proj, 'hw3', f'{filename[:-5]}-{method}.png')
    method = 'two-step-aff'
    save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')

    # Ipad: One-point method...
    H = compute_H_one_step(image_points)
    output_img = apply_homography(H, img, resize=True, output_dim=img.shape
        [:-1])
    method = 'one-step'
    save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')

    # Task-2: frame:
    physical_points = [
        (0, 0),
        (0, 10.8),
        (13, 10.8),
        (13, 0),
    ]
    # 13, 10.8
    image_points = [
        (143, 391),
        (220, 628),
        (460, 352),
        (276, 106),
    ]

    filename = 'frame.png'
    img = read_image('hw3', filename)

    # frame: point-to-point
    img_p2p, H = correct_distortion_using_point_to_point_correspondence(
        img,
        physical_points=physical_points,
        image_points=image_points,
        vectorize=True,
        resize=True,
        output_dim=img.shape[:-1],
        )

    method = 'point-to-point'
    save_image(img_p2p, 'hw3', f'{filename[:-5]}-{method}.png')

    # frame: two-point

    img_proj, img_aff, H_proj, H_aff, H_comb =
        correct_distortion_using_two_step_method(
        img, image_points, resize=True, output_dim=img.shape[:-1]
    )
    method = 'two-step-proj'
    save_image(img_proj, 'hw3', f'{filename[:-5]}-{method}.png')
    method = 'two-step-aff'
    save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')

```

```
561     # frame: one-point
562     H = compute_H_one_step(image_points)
563     output_img = apply_homography(H, img, resize=True, output_dim=img.shape
            [:-1])
564     method = 'one-step'
565     save_image(img_aff, 'hw3', f'{filename[:-5]}-{method}.png')
```

Listing 1: Example Python code